



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXÁMEN PARCIAL

PYTHON

Instrucciones:

- Lee cuidadosamente cada punto antes de programar.
- Cuentan con dos horas para realizarlo.
- Se puede disminuir puntaje de cada punto de acuerdo a cómo esté desarrollado.
- El examen se aprueba con 50 puntos.
- Solo se evaluarán: funciones, tuplas, listas, diccionarios, conjuntos, ciclos y condicionales.

1. Ejercicios de Python (100 puntos)

1. Ajuste de precios (8 pts)

Explicación: Se pide procesar una lista de precios aplicando reglas distintas según el rango de valores. Es un ejercicio de ciclos y condicionales. **Consigna:** Escribe una función `ajustar_precios(precios)` que devuelva una nueva lista ajustada según:

- Si $\text{precio} < 100$, aumentar 10%.
 - Si $100 \leq \text{precio} < 300$, dejar igual.
 - Si $\text{precio} \geq 300$, reducir 5%.
-

2. Limpieza de nombres (8 pts)

Explicación: Ejercicio de normalización de datos y eliminación de duplicados preservando el orden. **Consigna:** Implementa `limpiar_nombres(nombres)` que reciba una lista de strings, quite espacios, convierta a minúsculas y elimine duplicados.

3. Conteo por categorías (8 pts)

Explicación: Introducción al uso de diccionarios como acumuladores de conteo. **Consigna:** Dada una lista de tuplas (`producto`, `categoría`), implementa `conteo_categorias(pares)` que devuelva un diccionario `{categoría: cantidad}`.

4. Intersección y diferencia de conjuntos (8 pts)

Explicación: Ejercicio para practicar operaciones básicas con conjuntos a partir de listas con duplicados. **Consigna:** Escribe `comparar_listas(A,B)` que devuelva una tupla (`en_ambos`, `solo_A`, `solo_B`) con elementos en común y exclusivos.

5. Filtrar registros por condición (10 pts)

Explicación: Ejercicio de filtrado de diccionarios dentro de una lista, combinando ciclos, condicionales y manejo de claves inexistentes. **Consigna:** Implementa `filtrar_registros(registros, campo, umbral)` que devuelva solo los registros con `campo ≥ umbral`.

6. Máximo y mínimo con posiciones (10 pts)

Explicación: Se busca trabajar con recorridos manuales de listas, sin usar funciones built-in de `min` o `max`. **Consigna:** Define `extremos(nums)` que retorne (`min_val`, `pos_min`, `max_val`, `pos_max`) con índices de la primera aparición.

7. Unir inventarios (10 pts)

Explicación: Se necesita combinar inventarios parciales sumando cantidades por código. Ejercicio de acumulación en diccionarios. **Consigna:** Dadas dos listas de tuplas (`codigo`, `cantidad`), implementa `unir_inventarios(A,B)` que devuelva `{codigo: cantidad_total}`.

8. Ventanas contiguas válidas (10 pts)

Explicación: Se practica el concepto de ventanas deslizantes para optimizar cálculo de sumas de sublistas. **Consigna:** Implementa `contar_ventanas(nums, k, suma_max)` que cuente cuántas sublistas de tamaño k cumplen que su suma $\leq$ `suma_max`.

9. Re-mapear etiquetas (10 pts)

Explicación: Ejercicio de reemplazo condicionado usando un diccionario como mapa de traducción. **Consigna:** Implementa `remapear_etiquetas(datos, mapa)` que reemplace la clave "`etiqueta`" de cada registro según el diccionario de mapeo.

10. Tuplas de pares ordenados (8 pts)

Explicación: Uso de tuplas y ciclos anidados con condición sobre pares. **Consigna:** Escribe `pares_ordenados(nums)` que devuelva pares (a,b) con $a < b$ y suma par, sin repetir pares.

11. Palabras únicas y repetidas (6 pts)

Explicación: Ejercicio clásico de conteo de frecuencias para separar palabras únicas y repetidas. **Consigna:** Implementa `analizar_palabras(palabras)` que retorne una tupla (`unicas`, `repetidas`) según las frecuencias.

12. Compactar secuencias (8 pts)

Explicación: Ejercicio para detectar repeticiones consecutivas en una lista y resumirlas en tuplas. **Consigna:** Implementa `compactar(xs)` que devuelva una lista de tuplas (`valor`, `cantidad_consecutiva`).

Criterios de Corrección

- Correctitud y manejo de casos borde (listas vacías, claves ausentes, etc.).
- Uso de funciones, tuplas, listas, diccionarios, conjuntos, ciclos y condicionales (exclusivo).
- Claridad de código: nombres adecuados, no repetir lógica, retornos bien definidos.
- Eficiencia razonable (p.ej., actualización incremental en ventanas).